

木構造

1. はじめに：データ構造と「木」の概念

ITエンジニア、特に基本情報技術者（FE）試験を目指す方にとって、「アルゴリズムとデータ構造」は避けて通れない最重要科目です。プログラムの実行効率を決定づけるのは、コードの書き方以上に「データをどう保持するか」という戦略にあります。

「木構造」の定義と基本

木構造（ツリー構造）とは、データ要素である「ノード（節）」と、それらをつなぐ「エッジ（枝）」で構成される階層的なデータ構造です。最上位のノードを「ルート（根）」、末端のノードを「リーフ（葉）」と呼びます。

実社会のメタファー

文系の方でも直感的に理解できる例が、PCの「フォルダ構造」や「家系図」です。これらは「親ノードが複数の子ノードを持つ」という階層関係によって、情報の関連性を整理しています。

戦略的意義：なぜ木構造なのか（So What?）

配列のような線形構造での検索は、端から順に調べる「線形探索」となり、計算量が多くなります。これに対し、適切に構築された木構造は、一度の比較で探索範囲を劇的に絞り込むことができます。この「探索範囲の半減」こそが、計算機科学における高速化の真髄です。

次セクションからは、試験でも頻出する「二分木」のバリエーションを深掘りします。

2. 完全二分木（Complete Binary Tree）：整然とした構造の美学

二分木の中でも、すべてのレベルが満たされ、葉が左から順に詰まっている状態を「完全二分木」と呼びます。この構造は、計算機資源の管理において極めて高い安定性を誇ります。

完全二分木の定義

- 各ノードの子は最大2つ（左の子、右の子）。
- 上から順に、かつ同じ階層では左から順にノードが埋まっている。

構造的利点：メモリの最大活用

完全二分木は、ポインタを使わず「配列」で表現できるという大きなメリットがあります。2023年以降の**基本情報技術者試験（新疑似言語形式）**の基準に従うと、インデックス i のノードに対して以下の計算式で親子関係を導き出せます。

- 左の子： $2i + 1$
- 右の子： $2i + 2$
- 親ノード： $(i - 1) / 2$ （小数点以下切り捨て）

安定性の本質（So What?）

このように計算のみでアクセスできるため、メモリの空き領域（無駄）が発生しません。また、左右のバランスが取れていることで、木の高さを最小限（ $\log n$ 程度）に抑えられ、探索ステップ数を常に一定に保つことが可能になります。

3. 二分探索木（Binary Search Tree）：高速検索のメカニズム

「検索（サーチ）」という目的に特化したのが二分探索木（BST）です。

配置のルール

二分探索木では、以下のルールに従ってデータを配置します。

- 左の子 < 親ノード
- 親ノード < 右の子

検索効率と実行オーダー

このルールにより、一度の比較で探索範囲が半分になります。これはソースコンテキストにある「バイナリサーチ（二分探索）」の概念を木構造で実現したものであり、平均計算量は $O(\log n)$ と極めて高速です。

専門家の視点：不均衡という罠（So What?）

ここで注意すべきは最悪のケースです。例えば「1, 2, 3, 4, 5」とソート済みのデータを順に挿入すると、木は右側にだけ伸びる「偏った木（Degenerate Tree）」になります。この場合、構造は単なる線形リストと同じになり、計算量は一気に増加します。試験対策としては、「木がバランスしていること」の重要性をセットで理解しておきましょう。

4. 多分木（Multi-way Tree）：大規模データへの対応

メモリ上の処理だけでなく、データベース（DB）のような大規模システムでは、二分木の限界を超える「多分木」が活躍します。

多分木の構造

一つのノードから3つ以上の枝を出すことができる構造です。

項目	二分木	多分木（B-tree等）
枝の数	最大2つ	複数（m個）
木の高さ	高くなりやすい	劇的に低く抑えられる
主な用途	メモリ上の探索	DB、ファイルシステム

ディスクアクセスの壁を破る（So What?）

現代の計算機において、メインメモリへのアクセスに比べ、HDDやSSDといった「外部記憶装置」へのアクセス（ディスクI/O）は極めて低速です。多分木を採用して枝の数を増やすと、木の高さが低くなります。「木の高さ = ディスクアクセス回数」であるため、階層を圧縮することが、大規模データの高速処理における決定的な戦略となるのです。

5. ヒープ（Heap）：優先順位とソートの達人

親子の大小関係に厳格なルールを持たせた完全二分木が「ヒープ」です。

ヒープの条件

- 親ノードの値 $\geq$ 子ノードの値（最大ヒープの場合）
- ルートノードには、常にデータ全体の最大値（または最小値）が存在する。

リアルタイム処理への応用（So What?）

ヒープの真価は、全体をソートしなくても「常に最大のものを取り出せる」状態を維持できる点にあります。これは「**優先度付きキュー**」として、**OSのタスクスケジューリング**（緊急度の高い処理を優先する仕組み）に利用されています。未処理のタスクが大量にあっても、常に最優先のタスクを素早く特定できるインパクトは絶大です。

6. まとめ：適切なデータ構造の選択

本ガイドで学んだ各構造は、エンジニアが解決すべき「問い」への答えそのものです。

1. 完全二分木：配列実装によるメモリ効率とバランスの基本。
2. 二分探索木：動的なデータ管理と高速検索。ただし「偏り」に注意。
3. 多分木：ディスクI/Oを最小化し、大規模DBを支える戦略。
4. ヒープ：優先順位に基づいた、即時性の高い処理。

アルゴリズムの真髄は、コードを書く前段階の「適切なデータ構造の選択」にあります。試験の疑似言語問題でも、まずは「どのデータ構造が使われているか」を見極めてください。